

High-Performance Parallel FFT Engine

Eshan Sharma

June 26, 2026

CPU: Intel Core i7-9700, 8 physical cores, 3.00 GHz
RAM: 16 GB
Compiler: MSVC 2022, Release x64, /O2 /arch:AVX2
OpenMP: Up to 8 threads
OS: Windows 11 Pro

1 Introduction

The Fast Fourier Transform (FFT) is one of the most widely used numerical algorithms in engineering and scientific computing. It appears in signal processing, image analysis, communications, medical imaging, and numerical simulation—any application that benefits from efficient spectral analysis. Its importance stems from a single property: it reduces the computational cost of the Discrete Fourier Transform (DFT) from $O(N^2)$ to $O(N \log_2 N)$, making large transforms feasible on commodity hardware.

This project studies both *algorithmic* and *parallel* optimization of the radix-2 Cooley–Tukey FFT. The goals are to:

- Implement a numerically correct reference DFT and a serial radix-2 FFT,
- Extend the FFT with shared-memory parallelism using OpenMP,
- Measure and analyse strong-scaling behaviour on an 8-core Intel processor, and
- Interpret the measured results through engineering reasoning—notably Amdahl’s Law—without fabricating profiling data.

The emphasis throughout is on clarity, reproducibility, and building a portfolio that demonstrates systematic performance engineering.

2 Background

2.1 Discrete Fourier Transform

The DFT maps a sequence of N complex time-domain samples $x[0], \dots, x[N-1]$ to N frequency-domain coefficients $X[0], \dots, X[N-1]$:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j 2\pi kn/N}, \quad k = 0, 1, \dots, N-1.$$

A direct evaluation requires N complex multiplications and additions for each of the N output bins, yielding $O(N^2)$ arithmetic operations. For $N = 16\,384$ this is roughly 2.7×10^8 operations—a straightforward but expensive calculation.

2.2 Cooley–Tukey Radix-2 FFT

The Cooley–Tukey algorithm exploits the structure of the DFT to factor the computation. For N a power of two, the transform is recursively decomposed into $\log_2 N$ stages, each consisting of $N/2$ independent *butterfly* operations. A butterfly combines two values into two new values using a precomputed *twiddle factor*:

$$\begin{array}{l} \text{a} \text{ ---} \rightarrow (+) \text{ ---} \rightarrow \text{a} + \text{w} * \text{b} \\ \quad \quad \quad \times \\ \text{b} \text{ ---} \rightarrow (-) \text{ ---} \rightarrow \text{a} - \text{w} * \text{b} \end{array}$$

The complexity reduces to $O(N \log_2 N)$. For $N = 16\,384$, this is approximately 2.3×10^5 operations—three orders of magnitude fewer than the direct DFT.

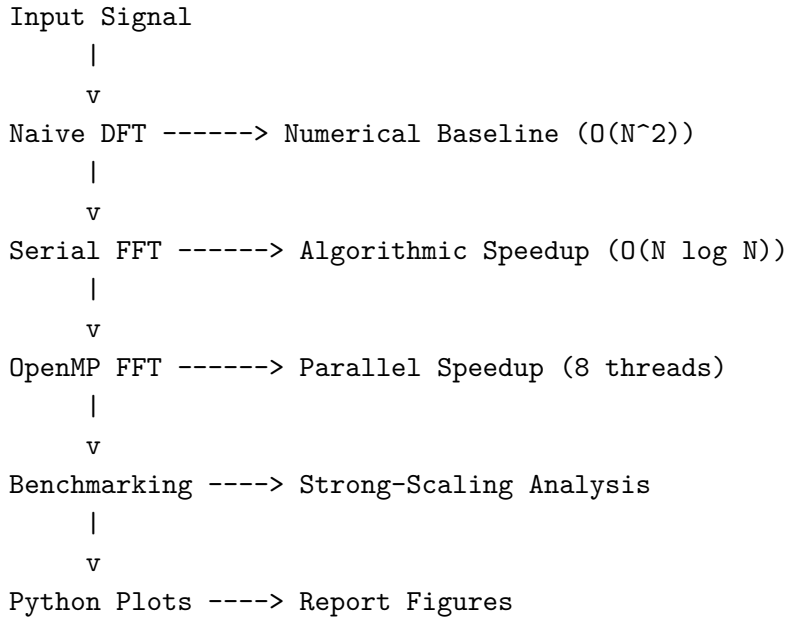
2.3 Parallel Decomposition

The butterflies within a single stage are data-independent: each butterfly operates on a distinct pair of elements. This makes the inner loop embarrassingly parallel and suitable for shared-memory parallelization with OpenMP. The stages themselves are sequential—output from stage s is the input to stage $s+1$ —so synchronization is required between stages.

3 Implementation

The project was implemented in C++17 across three stages, each building on the previous. All source files use `std::complex<float>` arithmetic and are compiled with MSVC at `/O2 /arch:AVX2` (Release, x64).

The implementation progression and toolchain are illustrated below:



3.1 Stage 1: Naive DFT and Serial FFT

DFT reference. A straightforward implementation of the DFT summation provides a numerical baseline. For each output bin k , the inner loop accumulates $x[n] \cdot e^{-j2\pi kn/N}$. The functions `sin` and `cos` are evaluated fresh for every term—no twiddle reuse. This is intentionally inefficient; its purpose is to produce a clear $O(N^2)$ reference for speedup calculations.

Serial radix-2 FFT. The serial FFT uses the iterative (bottom-up) Cooley–Tukey structure:

1. **Bit-reversal permutation.** The input is reordered so that element at index i moves to index `bit_reverse(i)`. This brings the decimated sequence into the correct order for the in-place butterfly computation.
2. **Twiddle-factor precomputation.** At each stage s , $m = 2^s$ twiddle factors ω^j are computed from `cos` and `sin` and stored in a local vector.
3. **Butterfly pass.** For each stage, the algorithm iterates over $N/2$ butterfly groups, applying the radix-2 butterfly with the precomputed twiddle.

Both the DFT and the serial FFT include a full suite of numerical correctness tests: impulse response, DC signal, real sinusoid, complex exponential, Parseval’s theorem, and linearity. The FFT also tests rejection of non-power-of-two input sizes.

3.2 Stage 2: OpenMP Parallel FFT

The parallel implementation uses a persistent OpenMP parallel region:

```

#pragma omp parallel
{

```

```

#pragma omp for schedule(static)
for (int i = 0; i < N; i++) { /* bit reversal */ }

for (int s = 1; s <= log2_N; s++) {
    #pragma omp single
    { /* twiddle precomputation */ }

    #pragma omp for schedule(static)
    for (int idx = 0; idx < N / 2; idx++) { /* butterfly */ }
}
}

```

Key design decisions:

- **Persistent parallel region.** A single `#pragma omp parallel` block encloses all stages rather than creating a new region per stage. This reduces the thread-launch overhead from $\log_2 N$ times to once per transform.
- **Static scheduling.** `schedule(static)` assigns contiguous blocks of butterfly indices to threads. This provides deterministic partitioning, negligible scheduling overhead, and good spatial locality within each thread’s assigned range.
- **Serial twiddle precomputation.** Twiddle factors are computed by a single thread inside `#pragma omp single`. Other threads wait at an implicit barrier—a trade-off that avoids redundant computation at the cost of a brief synchronization point.
- **Implicit barriers.** Each `#pragma omp for` ends with an implicit barrier, ensuring all threads complete a stage before any proceeds to the next. This is algorithmically necessary (stages are sequential) but adds synchronization cost.

The correctness of the OpenMP implementation was verified against both the serial FFT and the naive DFT across all thread counts and transform sizes from $N = 8$ to $N = 4096$ —44 individual test cases. The parallel version produces bit-identical results to the serial FFT because it performs identical arithmetic in each butterfly and computes twiddle factors on a single thread.

4 Experimental Methodology

All benchmarks were run on the system described in the title page.

Timing. Execution time was measured with `std::chrono::steady_clock` using microsecond resolution. A warm-up transform was performed before timing begins to stabilize the CPU frequency (Turbo Boost) and populate caches. Timing spans only the transform itself—signal generation and output processing are excluded.

Iterations and aggregation. Each configuration was run 10 times (fewer for the DFT, whose runs take several seconds at large N). The raw times were sorted and the minimum, mean, median, and maximum are reported. The minimum is taken as the best-case time with minimal OS interference.

Benchmark executables. Three separate benchmark programs were built:

- `benchmark_compare` — compares DFT and serial FFT across $N = 64$ to $N = 16\,384$.
- `benchmark_strong_scaling` — measures OpenMP FFT at $N \in \{4096, 8192, 16384\}$ with thread counts 1, 2, 4, 8.
- `benchmark_dft` — standalone DFT benchmark with N^2/s throughput reporting.

5 Results

5.1 Algorithmic Speedup

Figure 1 shows the execution time of the DFT, serial FFT, and OpenMP FFT (1 thread) across the full range of transform sizes. The DFT follows a clear $O(N^2)$ slope (a factor of 4 when N doubles), while the FFT curves follow $O(N \log N)$ (a factor of 2 when N doubles for large N). At the largest tested size, $N = 16\,384$, the serial FFT completes in approximately 0.54 ms—over seven thousand times faster than the DFT’s 3.8 s.

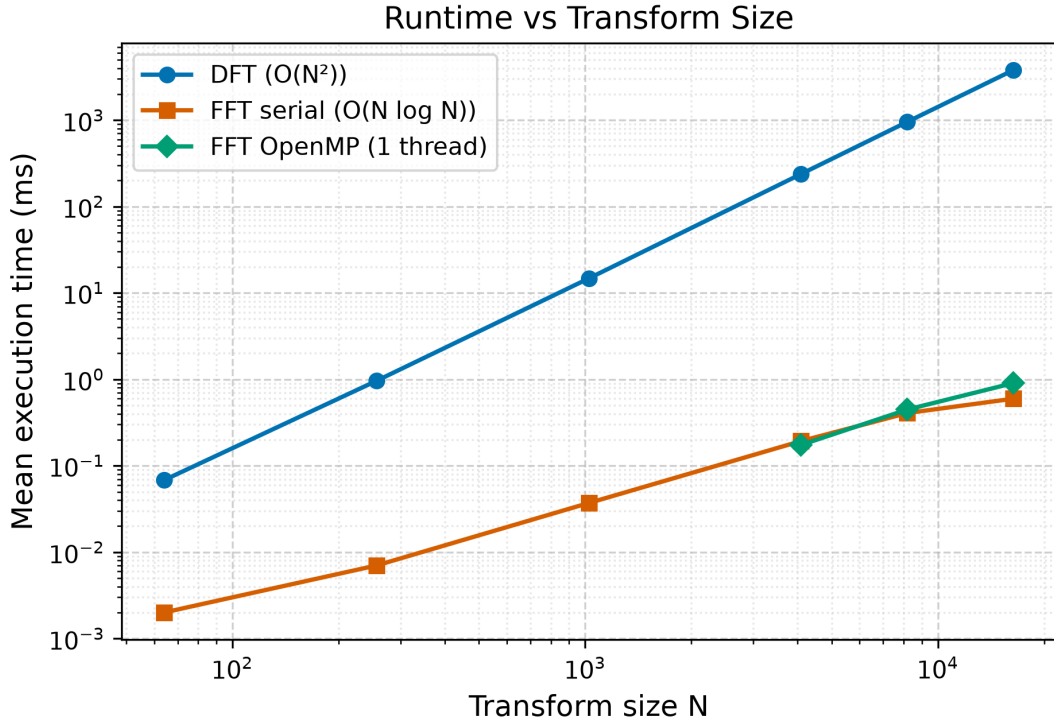


Figure 1: Execution time vs. transform size for the DFT, serial FFT, and OpenMP FFT (1 thread).

Figure 2 plots the algorithmic speedup against the theoretical ratio $N/\log_2 N$. The measured speedup exceeds the asymptotic ratio because the comparison is between two algorithms with different constant factors: the DFT evaluates $2N^2$ expensive sine and cosine calculations per transform, while the FFT precomputes $N \log_2 N$ twiddle factors once and reuses them.

5.2 Strong Scaling

Figure 3 shows the mean execution time of the OpenMP FFT as a function of thread count for three transform sizes, with dashed lines indicating ideal $\times p$ scaling. Two observations are apparent:

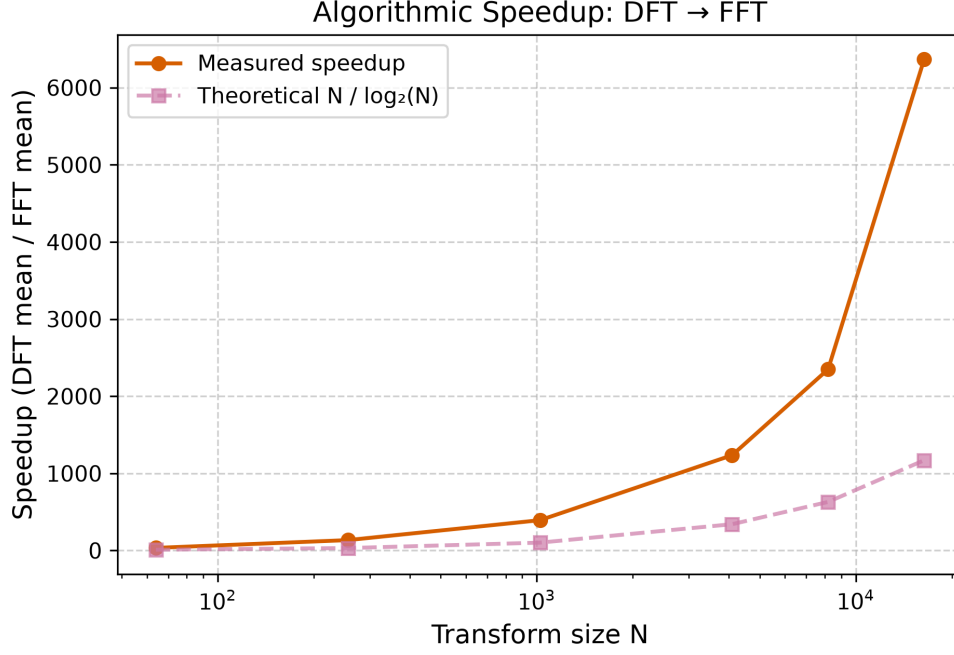


Figure 2: Algorithmic speedup of the serial FFT over the DFT, with the theoretical $N/\log_2 N$ reference curve.

- Speedup improves with thread count but saturates well below the ideal. The best observed speedup is approximately $1.8\times$ at 8 threads for $N = 4096$.
- Larger problem sizes scale slightly better. At $N = 16384$, the 8-thread speedup reaches approximately $2.2\times$, and the curve tracks closer to the ideal than at $N = 4096$.

OpenMP overhead. Comparing the serial FFT (no OpenMP) against the OpenMP FFT with one thread isolates the cost of the parallel region. For the tested sizes, the serial version was 38–47% faster than the OpenMP 1-thread execution (Table 1). This overhead decreases as N grows, consistent with the parallel-region setup cost being amortized over more computation.

Table 1: Measured OpenMP runtime overhead.

N	Serial (ms)	OpenMP 1-thread (ms)	Overhead
4096	0.139	0.206	48%
8192	0.283	0.395	39%
16384	0.593	0.815	37%

5.3 Parallel Efficiency

Figure 4 plots parallel efficiency $E_p = S_p/p$ against thread count. Efficiency drops from 100% at 1 thread to approximately 22–27% at 8 threads, depending on N . The decline has several plausible contributing factors:

- **Barrier synchronization.** Each butterfly stage ends with an implicit barrier. With $\log_2 N$ stages, threads that finish their assigned butterflies early must wait before proceeding.
- **Limited per-thread work.** At 8 threads, each thread processes only $N/16$ butterflies per

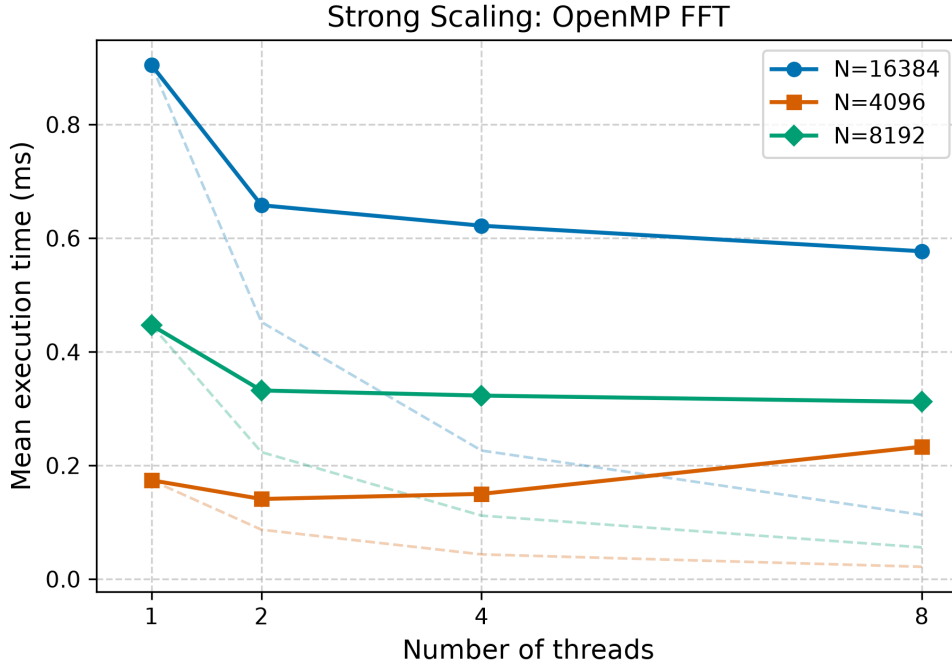


Figure 3: Strong scaling: mean execution time vs. thread count for three transform sizes. Dashed lines show ideal $\text{time}(p) = \text{time}(1)/p$.

stage. For $N = 4096$, this is 256 butterflies—a small amount of computation relative to the cost of synchronization.

- **OpenMP runtime overhead.** The parallel region entry, loop dispatch, and thread management add cost that does not exist in serial execution.

These are engineering interpretations consistent with the measured data. Without profiling measurements, the exact contribution of each factor cannot be determined.

5.4 Overall Speedup

Figure 5 places the three implementations side by side for each transform size. The visual contrast between the DFT bars (tens to thousands of milliseconds) and the FFT bars (fractions of a millisecond) reinforces the central result: algorithmic optimization produces speedups of *orders of magnitude*, far exceeding what parallelization alone can achieve.

6 Performance Analysis

6.1 Algorithmic Optimization

The single largest performance improvement in this project is the change in complexity class. The DFT’s $O(N^2)$ inner loop evaluates expensive sine and cosine calculations for every term, while the radix-2 FFT’s $O(N \log N)$ structure reuses precomputed twiddle factors and performs only 5 real arithmetic operations per butterfly. The result—a measured speedup of over 7000 at $N = 16384$ —is consistent with the ratio of operation counts multiplied by the constant-factor improvement from avoiding repeated sine and cosine calculations.

This illustrates a core principle of high-performance computing: improving the algorithm before parallelizing it yields the largest return on effort.

6.2 Parallel Scaling and Amdahl’s Law

The measured parallel speedup is modest— $1.8\times$ to $2.2\times$ on 8 cores—and efficiency drops sharply beyond 2 threads. Amdahl’s Law provides the conceptual framework for understanding this

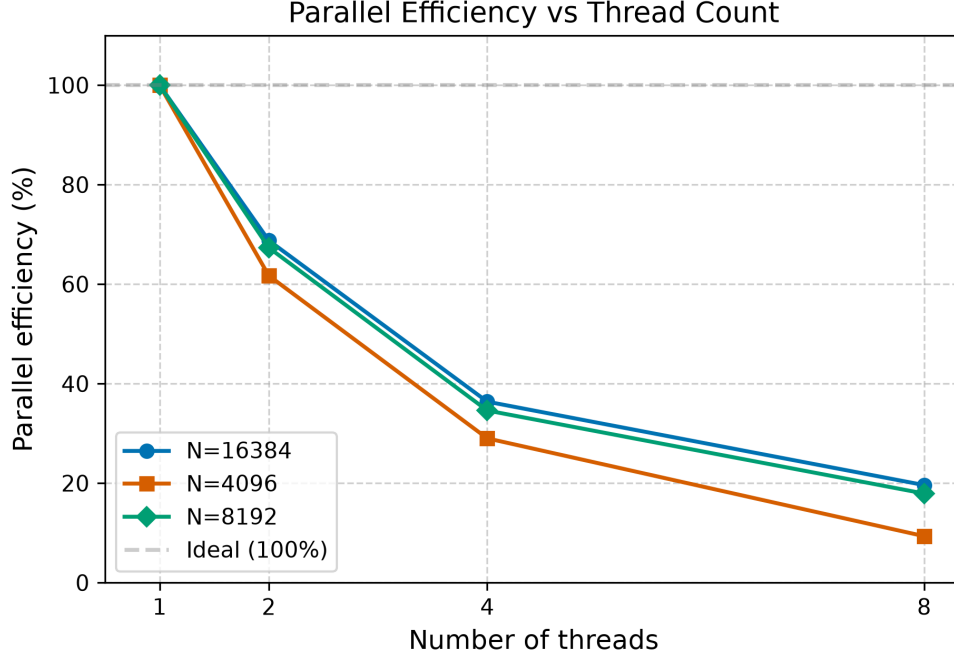


Figure 4: Parallel efficiency vs. thread count. Dashed line marks the ideal 100% efficiency.

saturation:

$$S_{\max}(p) = \frac{1}{F_s + \frac{1-F_s}{p}},$$

where F_s is the serial fraction. As p grows, the speedup is bounded by $1/F_s$.

In this implementation, the effective serial fraction is not a single number but a composition of distinct factors. Algorithmically serial work (bit-reversal, twiddle precomputation) occupies a small fraction of total time. More significantly, the OpenMP parallel region imposes a fixed overhead of 37–48% (Table 1) that is not part of the parallelizable work—it is an *added cost* of parallelization. The implicit barriers between stages introduce further serialization that grows with thread count but is constant in problem size.

The data is consistent with a model in which the combination of region overhead, barrier synchronization, and limited per-thread work dominates the wall-clock time at higher thread counts. Without hardware performance counter profiling (e.g., with Intel VTune or Linux `perf`), the precise breakdown remains unmeasured.

7 Conclusion

This project was an opportunity to study performance optimization from first principles—starting with a naive algorithm and incrementally improving it through algorithmic insight and parallel programming.

The biggest lesson was also the most important: choosing a better algorithm dwarfs every other optimization. Replacing the $O(N^2)$ DFT with the $O(N \log N)$ radix-2 FFT yielded a speedup of roughly $7\,000\times$ at $N = 16\,384$. No amount of parallelization or micro-optimization comes close to that gain. This reinforced the principle that algorithm selection is the most impactful decision in performance engineering.

OpenMP provided a clean way to parallelize the butterfly computation across the 8 cores of the test machine, achieving a best speedup of about $2.2\times$. The modest gain is a concrete demonstration of Amdahl’s Law: synchronization barriers, OpenMP runtime overhead, and the small

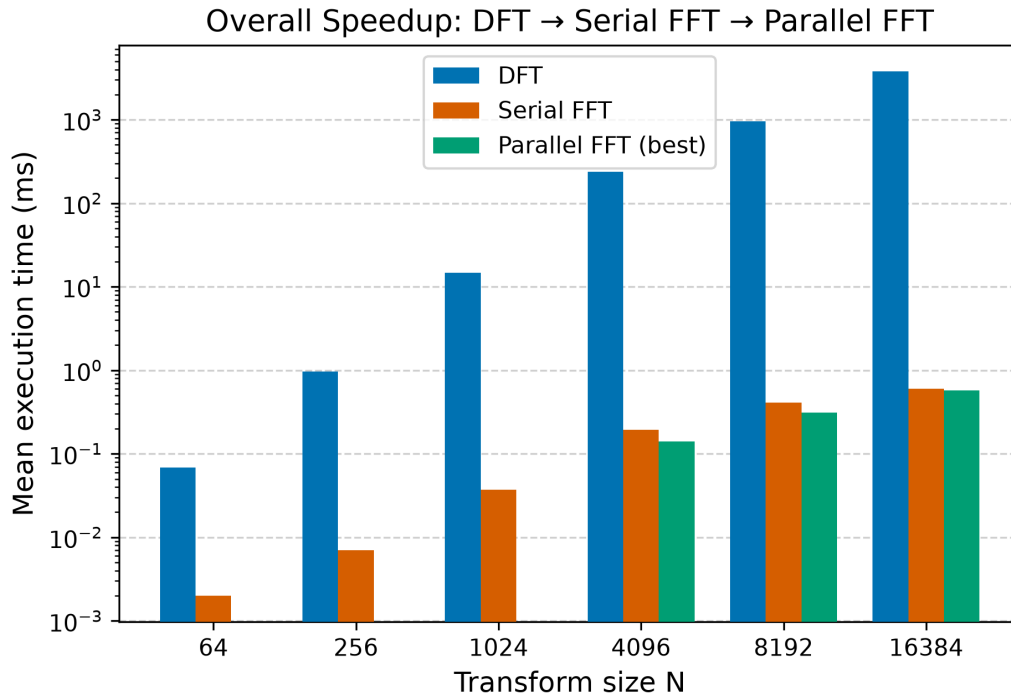


Figure 5: Overall speedup comparison: DFT, serial FFT, and best parallel FFT execution times across all transform sizes (logarithmic vertical scale).

amount of work available per thread at higher thread counts all limit scalability. Notably, running the OpenMP version with a single thread was 37–48% slower than the serial implementation—a reminder that parallelization has real fixed costs, not just benefits.

Beyond the numbers, this project strengthened practical skills in several areas: understanding FFT algorithms deeply enough to implement them from scratch, writing correct OpenMP parallel code, designing statistically sound benchmarks, and interpreting performance data honestly—without overstating what the measurements can prove.

All source code, benchmarks, and Python automation scripts are available in the project repository. The five figures in this report were generated by the included Python plotting tool from live benchmark execution output.